Convolutional Neural Networks (CNNs)

EEP 596 Computer Vision: Classical and Deep Methods (Autumn 2026)

UNIVERSITY of WASHINGTON

Stan Birchfield

NVIDIA

1

Shifting gears

We've been doing this



Now, let's do this



2

First, let's be clear

- We are only talking about **supervised learning**
  (not unsupervised, semi-supervised, self-supervised, weakly supervised, reinforcement)
- We are only concerned with simple, **feedforward networks**
  (not recurrent)
- We are only looking at **CNNs**
  (not Transformers)
- We are only concerned with **single RGB image** as input
  (not RGBD, video, language, speech, multimodal)

3

The Problem

- We have some classes / categories
- Given an image, we want to classify the image
- Is it a dog or a cat?
- Easy: Train a neural network…



4

The Solution

- Step 1: Collect a dataset of images
  - Label the images
  - We will group the images into random mini-batches (e.g., 32 images per mini-batch)
- Step 2: Create a neural network
- Step 3: Train the network
- Now, we can apply the network to any image!
  (even to images outside the dataset – *generalization*)

5

How to get the dataset?

- Easy way: Download an existing classic dataset
  - MNIST
  - CIFAR-10
  - ImageNet-1k
- Or:
  - Find and download a non-standard dataset
  - Download / acquire your own images and label them

6

How to create the network?

- Easy way: Use an existing classic architecture / backbone
- Or:
  - Use a non-standard architecture / mix-and-match / tweak
  - Create your own

7

How to use the network?

Simply do this:

- Forward pass
  Send the image to the network, get the output

(Of course, this only works *after* the network has been trained)

8

How to train the network?

Repeat for each mini-batch:

1. Augmentation
   Randomly perturb images (crop, rotate, noise, etc.)
2. Forward pass
   Send the images to the network, get the output
3. Compute loss / error
   Compare the output to ground truth
4. Optimize
   Propagate the error backward (backpropagation) to compute derivatives
5. Update weights
   Use the derivatives to change the network weights

9

Now let's try it out…

DEEP LEARNING WITH PYTORCH: A 60 MINUTE BLITZ

Author: Soumith Chintala

We will jump straight to …

TRAINING A CLASSIFIER

https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html

10

The Steps

Training an image classifier

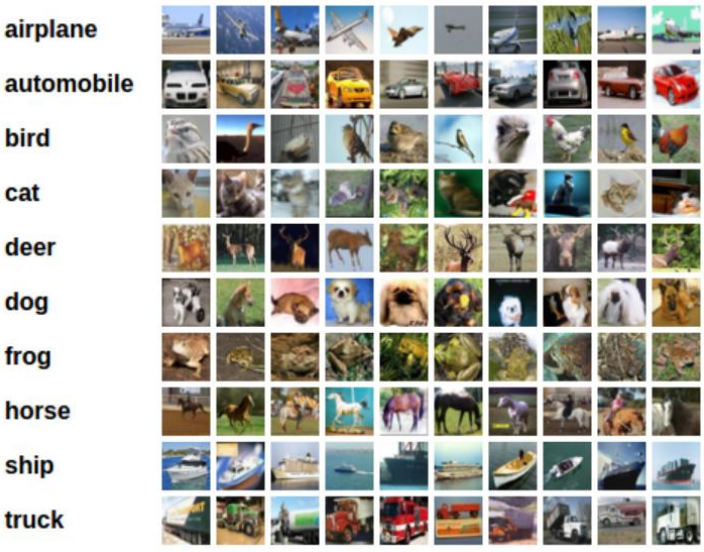
We will do the following steps in order:

1. Load and normalize the CIFAR10 training and test datasets using `torchvision`
2. Define a Convolutional Neural Network
3. Define a loss function
4. Train the network on the training data
5. Test the network on the test data

11

CIFAR-10

- CIFAR-10
- 50k images
- 10 categories
- image size:
  3 x 32 x 32



Note: It is just as easy to classify 10 categories as it is to classify 2

12

Load dataset

- `torchvision` is a package with **dataloaders** and **transforms**

```
import torch
import torchvision
import torchvision.transforms as transforms
```

Why this?

Data augmentation

- Define transform, load dataset

```
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])

batch_size = 4

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                        download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch_size,
                                          shuffle=True, num_workers=2)
```

These are chained together

13

Visualize dataset (for fun)

```
classes = ('plane', 'car', 'bird', 'cat',
           'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
```

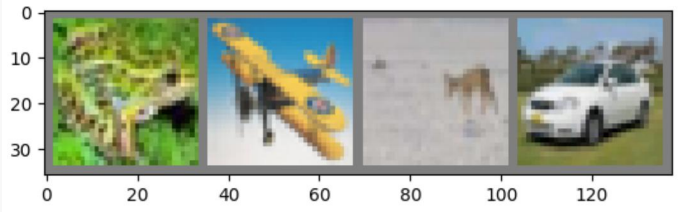
```
import matplotlib.pyplot as plt
import numpy as np

# functions to show an image


def imshow(img):
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()


# get some random training images
dataiter = iter(trainloader)
images, labels = next(dataiter)

# show images
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join(f'{classes[labels[j]]:5s}' for j in range(batch_size)))
```



Out:

```
frog  plane deer  car
```

14

Define and create the network

Define variables

Specify behavior

Create network

```
import torch.nn as nn
import torch.nn.functional as F

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1) # flatten all dimensions except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x


net = Net()
```

C_{in} C_{out} kernel width

Notice how the dimensions match?

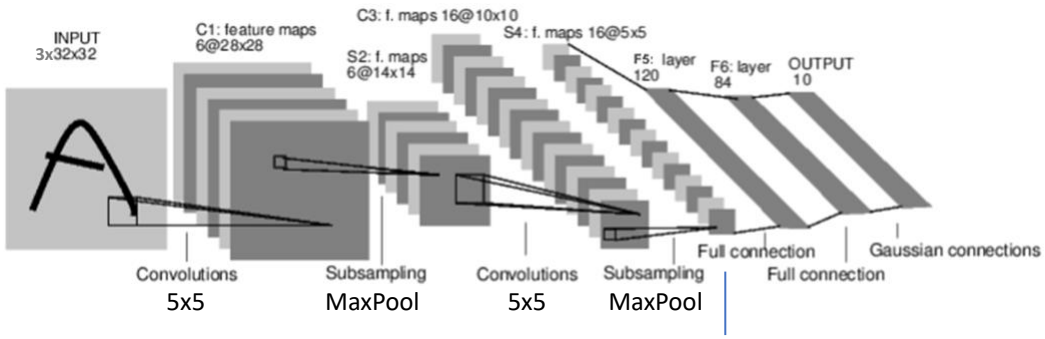
Where do these numbers come from?

Image is 5x5

15

Can you draw the network?

This is AlexNet



There are 16*5*5 = 400 pixels total

Nouns

```
self.conv1 = nn.Conv2d(3, 6, 5)
self.pool = nn.MaxPool2d(2, 2)
self.conv2 = nn.Conv2d(6, 16, 5)
self.fc1 = nn.Linear(16 * 5 * 5, 120)
self.fc2 = nn.Linear(120, 84)
self.fc3 = nn.Linear(84, 10)
```

Verbs

```
x = self.pool(F.relu(self.conv1(x)))
x = self.pool(F.relu(self.conv2(x)))
x = torch.flatten(x, 1)
x = F.relu(self.fc1(x))
x = F.relu(self.fc2(x))
x = self.fc3(x)
```

Draw tensor shape at each stage

16

Define loss function and optimizer

Remember:

- We have to calculate the loss
- We have to update the weights

```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```

- We are ignoring the details under the hood (for now) …

17

Train the network

```
for epoch in range(2):  # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999:    # print every 2000 mini-batches
            print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 2000:.3f}')
            running_loss = 0.0

print('Finished Training')
```

Augmentation

Forward pass
Calculate loss
Backprop to get gradients
Update weights

Out:

```
[1,  2000] loss: 2.144
[1,  4000] loss: 1.835
[1,  6000] loss: 1.677
[1,  8000] loss: 1.573
[1, 10000] loss: 1.526
[1, 12000] loss: 1.447
[2,  2000] loss: 1.405
[2,  4000] loss: 1.363
[2,  6000] loss: 1.341
[2,  8000] loss: 1.340
[2, 10000] loss: 1.315
[2, 12000] loss: 1.281
Finished Training
```

Save the weights after training:

```
PATH = './cifar_net.pth'
torch.save(net.state_dict(), PATH)
```

18

Some details



```
# forward + backward + optimize
outputs = net(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
```

1. Forward pass
   network performs conv, pooling, linear, activation
2. Calculate loss

```
loss = (prediction - labels).sum()
```

3. Backprop
   we will discuss this later
4. Update weights

```
learning_rate = 0.01
for f in net.parameters():
    f.data.sub_(f.grad.data * learning_rate)
```

implements `weight = weight - learning_rate * gradient`

19

Test the network on a single image

- Simply do this:

```
input = torch.randn(1, 3, 32, 32)
out = net(input)
print(out)
```

- Note:

`torch.nn` only supports mini-batches.

If you have a single sample, just use `input.unsqueeze(0)` to add a fake batch dimension.

- Here's the output:

Out:

```
tensor([[ 0.1453, -0.0590, -0.0065,  0.0905,  0.0146, -0.0805, -0.1211, -0.0394,
         -0.0181, -0.0136]], grad_fn=<AddmmBackward0>)
```

Ignore this for now, used for backprop

20

Test the network

```
# prepare to count predictions for each class
correct_pred = {classname: 0 for classname in classes}
total_pred = {classname: 0 for classname in classes}

# again no gradients needed
with torch.no_grad():
    for data in testloader:
        images, labels = data
        outputs = net(images)
        _, predictions = torch.max(outputs, 1)
        # collect the correct predictions for each class
        for label, prediction in zip(labels, predictions):
            if label == prediction:
                correct_pred[classes[label]] += 1
            total_pred[classes[label]] += 1


# print accuracy for each class
for classname, correct_count in correct_pred.items():
    accuracy = 100 * float(correct_count) / total_pred[classname]
    print(f'Accuracy for class: {classname:5s} is {accuracy:.1f} %')
```

Disable gradient computation

arg max

Out:

```
Accuracy for class: plane is 37.9 %
Accuracy for class: car   is 62.2 %
Accuracy for class: bird  is 45.6 %
Accuracy for class: cat   is 29.2 %
Accuracy for class: deer  is 50.3 %
Accuracy for class: dog   is 45.9 %
Accuracy for class: frog  is 60.1 %
Accuracy for class: horse is 70.3 %
Accuracy for class: ship  is 82.9 %
Accuracy for class: truck is 63.1 %
```

21